Anonymous classes are still classes and support all the features of regular classes. Member declaration works the same way, accessing static and non-static members works the same way, and so on. We will look at inheritance, interfaces, and traits later in the chapter, and all of those features work with anonymous classes too.

Class (Not Object) Context

In the previous section, our `QuoteBot` class didn't actually do anything until we created an object. Before we could make calls to the class methods, we needed an instance to work with – created with the keyword `new`. There is a mechanism, however, to access class methods and properties without instantiation. To do this we use a keyword you might remember from Chapter 3, "Functions": `static`.

Static Members

Properties accessed (or methods executed) statically are said to be running in class context, rather than the context of an instantiated object.

The syntax and rules for class context code are slightly different from those for objects. Let's change our `QuoteBot` class a little and see how it all works:

```
class QuoteBot
{
    public static $quoteStash = [
        "I'm a writer, you monsters! I create!",
        "I'll show you the life of the mind!"
    ];
}
```

With the `quoteStash` publicly accessible again, let's try to access it from an object instance:

```
$quoteBot = new QuoteBot();
print $quoteBot->quoteStash[1];
```

```
Output:
Notice: Accessing static property QuoteBot::$quoteStash as non static
Warning: Undefined property: QuoteBot::$quoteStash
Warning: Trying to access array offset on value of type null
```